Smart Agricultural Production Optimization Engine

Project Description

The Smart Agricultural Production Optimization Engine (OptiCrop) is a Machine Learning-based web application that helps farmers and agricultural stakeholders identify the most suitable crop based on soil nutrients and environmental conditions. The system predicts the best crop using parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH value, and rainfall.

The application is developed using Python, Flask, Scikit-learn, Pandas, and NumPy. A trained Random forest model, analyzes the user inputs and provides accurate crop recommendations to improve productivity and optimize agricultural production. The user-friendly, multi-page interface (Home, About, and Predict Crop) allows users to enter field conditions and instantly receive prediction results.

The project is deployed globally on Render, making it accessible through any web browser without additional software installation:

- Live URL: https://opticrop-u9b8.onrender.com
- Local URL (development): http://localhost:5000

By providing data-driven crop recommendations, OptiCrop supports better farming decisions, efficient resource utilization, and sustainable agricultural practices.

Scenarios

Scenario 1: Crop Recommendation

A farmer enters soil parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall. The system analyzes the inputs using the trained machine learning model and recommends the most suitable crop for cultivation.

Scenario 2: First-Time User

A new user accesses the deployed Flask application through the web browser. After entering the required agricultural parameters on the Find Your Crop page, the application instantly predicts the best crop and displays the recommendation in an easyto-understand format without requiring any login.

Scenario 3: Decision Support for Better Farming

An agricultural advisor compares different soil and weather conditions by entering multiple sets of values. The application provides crop recommendations for each case,

helping farmers make informed decisions that improve productivity and reduce the risk of unsuitable crop selection.

Technical Architecture

OptiCrop is built using Flask as the web application framework and Scikit-learn for machine learning. Users provide agricultural inputs through the Flask interface. Then processes them using the trained model.pkl model to predict the most suitable crop. The predicted crop recommendation is then displayed instantly to the user on a styled result page. The application is deployed on Render, making it accessible through any web browser without requiring installation.

Architecture Diagram

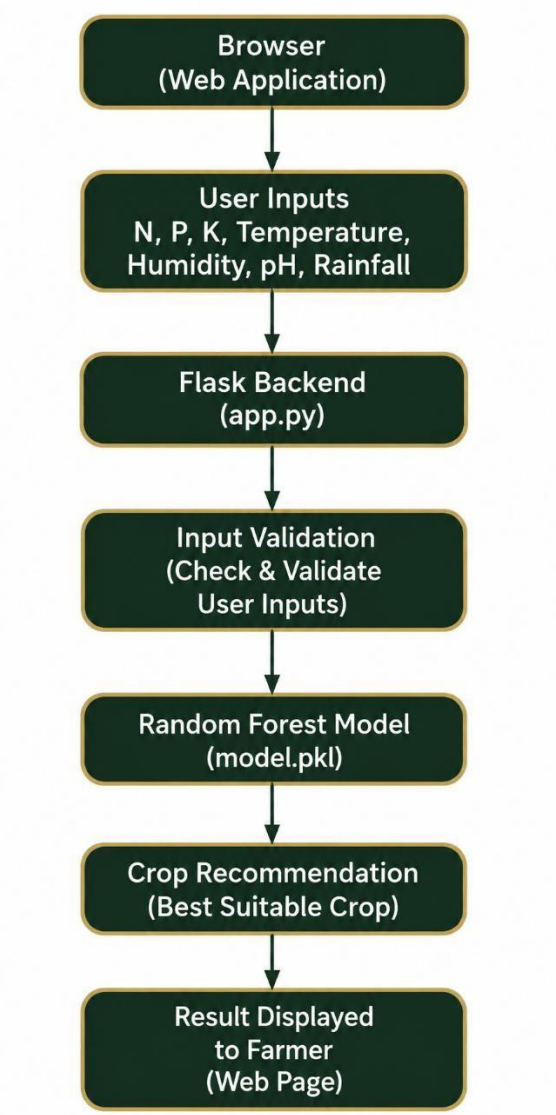

Figure 1.1: Data flow from browser input to crop recommendation.

Prerequisites & Technology Stack

- Python 3.10 or above
- Flask — web application framework ● Pandas, NumPy — data handling
- Matplotlib, Seaborn — visualization
- Scikit-learn — machine learning
- Pickle — model persistence
- Git & GitHub — version control
- Render — cloud deployment
- Internet connection (for accessing the deployed application)

Dataset Collection & Environment Setup

- Source: Kaggle — Smart Agricultural Production Optimizing Engine (chitrakumari25)
- Size: 2,200 rows, 22 crop classes, 100 samples per class (perfectly balanced)
- Features: N, P, K, temperature, humidity, ph, rainfall
- Target: label (crop name)
- Supported crops: apple, banana, blackgram, chickpea, coconut, coffee, cotton, grapes, jute, kidneybeans, lentil, maize, mango, mothbeans, mungbean, muskmelon, orange, papaya, pigeonpeas, pomegranate, rice, watermelon

Steps: install Python (3.10+), create a virtual environment, install dependencies using pip install -r requirements.txt, load the dataset using Pandas, and verify there are zero missing values across all columns.

Exploratory Data Analysis

Univariate Analysis

Each of the 7 numeric features was examined individually to understand its distribution shape, range, and modality — informing decisions about scaling and model choice.

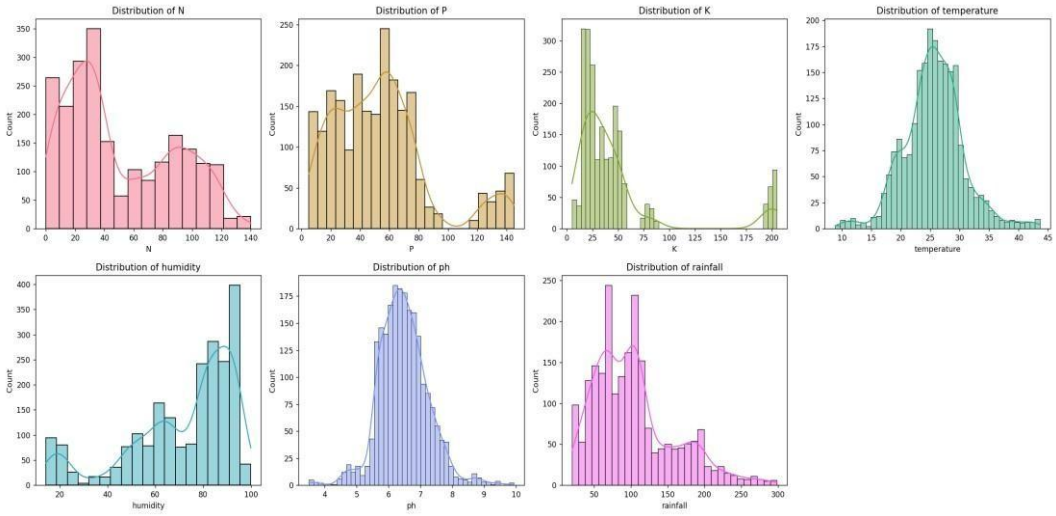

Figure 2.1: Distribution of all 7 agricultural features.

Bivariate Analysis

The plot below shows humidity plotted against crop type, revealing that different crops cluster around distinct humidity ranges — rice and jute both require consistently high humidity, while chickpea and mothbeans tolerate much lower humidity.

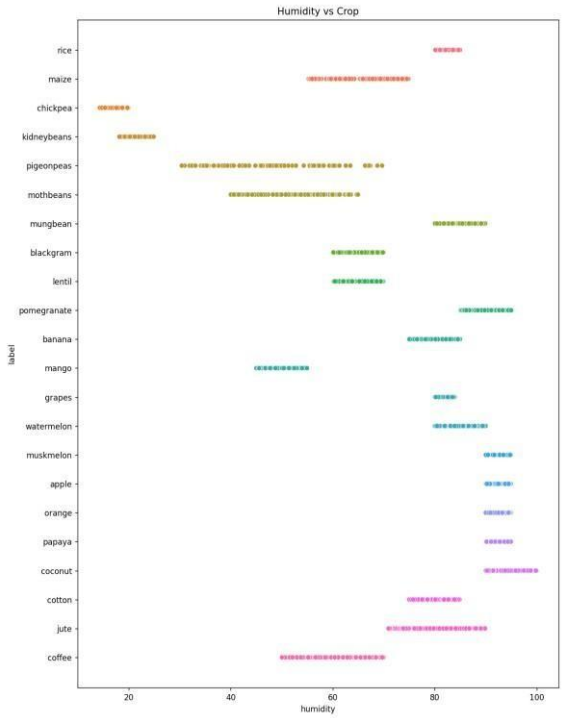

Figure 2.2: Humidity vs Crop — crop-specific humidity clustering.

Multivariate Analysis

A count plot confirms the class balance across all 22 crops — each crop has exactly 100 samples, validating that accuracy is an appropriate evaluation metric.

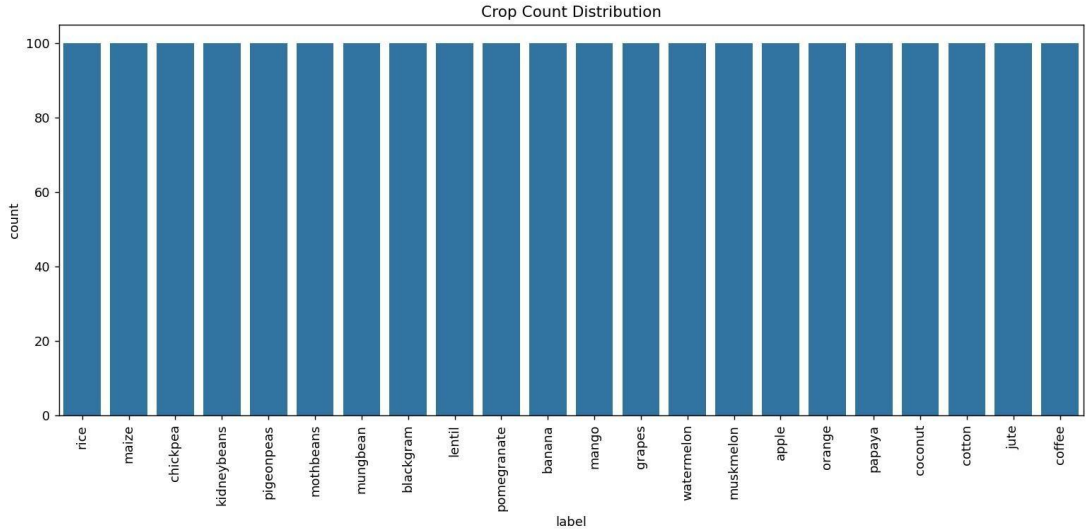

Figure 2.3: Crop count distribution — confirming perfect class balance.

Data Pre-processing

- Null check: confirmed zero missing values across all columns — no imputation required.
- Outlier review: IQR-based outlier counts were reviewed per feature (P: 138, K: 200, temperature: 86, humidity: 30, ph: 57, rainfall: 100), retained since they reflect legitimate crop-specific nutrient needs.
- Seasonal crop extraction: crops grouped into Summer, Winter, and Rainy categories using threshold rules on temperature, humidity, and rainfall.
- Train/test split: 80/20, stratified sampling to preserve class balance.

Random Forest

A Random Forest classifier was trained using 80% of the crop recommendation dataset and evaluated on the remaining 20%. Since Random Forest is a tree-based algorithm, feature scaling was not required. The model was trained directly on the original soil and environmental parameters (Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall). During prediction, the Flask application accepts user inputs, performs basic input validation, and passes the values directly to the trained model to recommend the most suitable crop. **Model Evaluation**

- Overall Accuracy: 97.3% on the held-out test set
- Near-perfect precision/recall (0.95-1.00): apple, banana, chickpea, grapes, kidneybeans, mungbean, pomegranate, watermelon
- Weakest performance: rice (recall 0.80) and jute (precision 0.83) — both require high rainfall and humidity, causing feature overlap that a linear decision boundary struggles to fully separate

Since all 22 classes are perfectly balanced (100 samples each), accuracy is a statistically fair metric for this evaluation.

ML Workflow Diagram

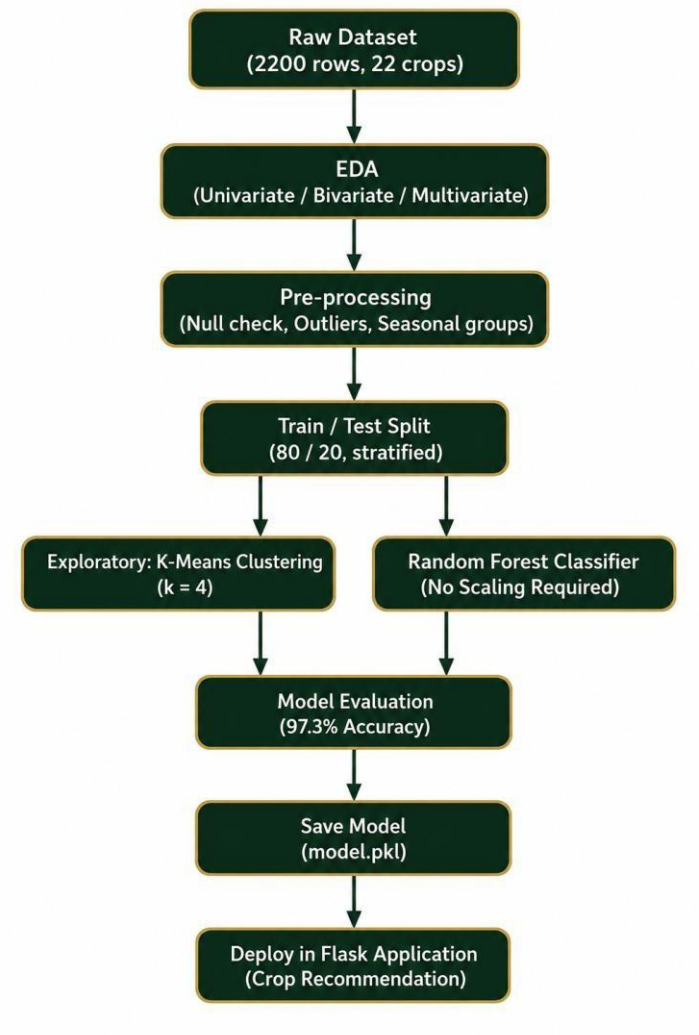

Figure 3.1: End-to-end machine learning workflow, from raw data to saved model.

Flask Application

The Flask backend (app.py) loads the trained machine learning model (model.pkl) once during application startup. The application provides four routes: **Home**, **About**, **Predict Crop**, and a **POST** route (/predict) that receives the user's soil and environmental parameters, performs basic input validation, predicts the most suitable crop using the trained Random Forest model, and displays the recommendation on the result page.

Folder Structure

```
OptiCrop/
├── app.py
├── requirements.txt
├── README.md
├── model/
│   └── crop_model.pkl
├── dataset/
│   └── Crop_recommendation.csv
├── notebooks/
│   └── OptiCrop.ipynb
├── static/
│   ├── css/
│   │   └── style.css
│   ├── images/
│   │   ├── about.jpg
│   │   ├── favicon.png
│   │   ├── hero.jpg
│   │   ├── logo.png
│   │   └── predict.png
│   └── js/
│       └── script.js
└── templates/
    ├── base.html
    ├── home.html
    ├── about.html
    ├── predict.html
    └── result.html
```

Home & About Page

The Home page introduces OptiCrop with a modern responsive interface, project overview, and a call-to-action leading to the prediction page. The page highlights all 7 input parameters the model uses.

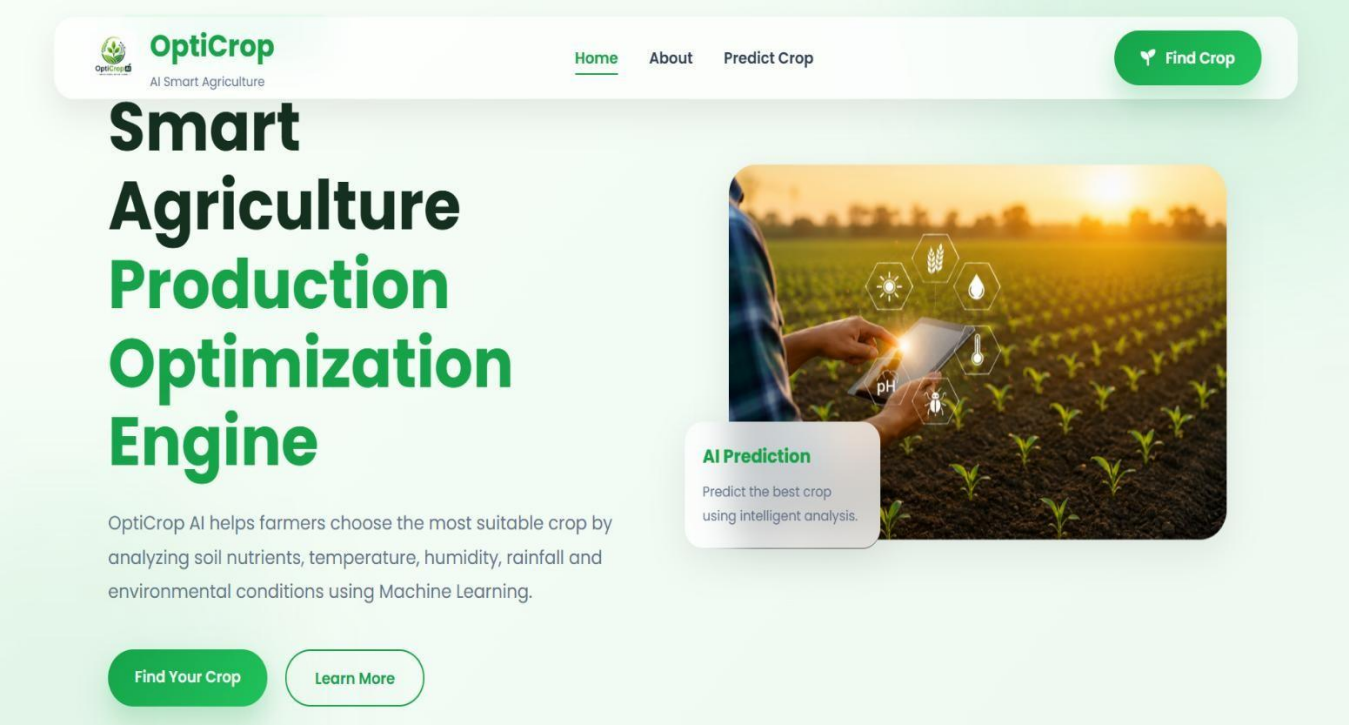

Figure 4.1: Home page.

The About page describes the purpose of OptiCrop and the three groups of users it supports: farmers, researchers, and policymakers.

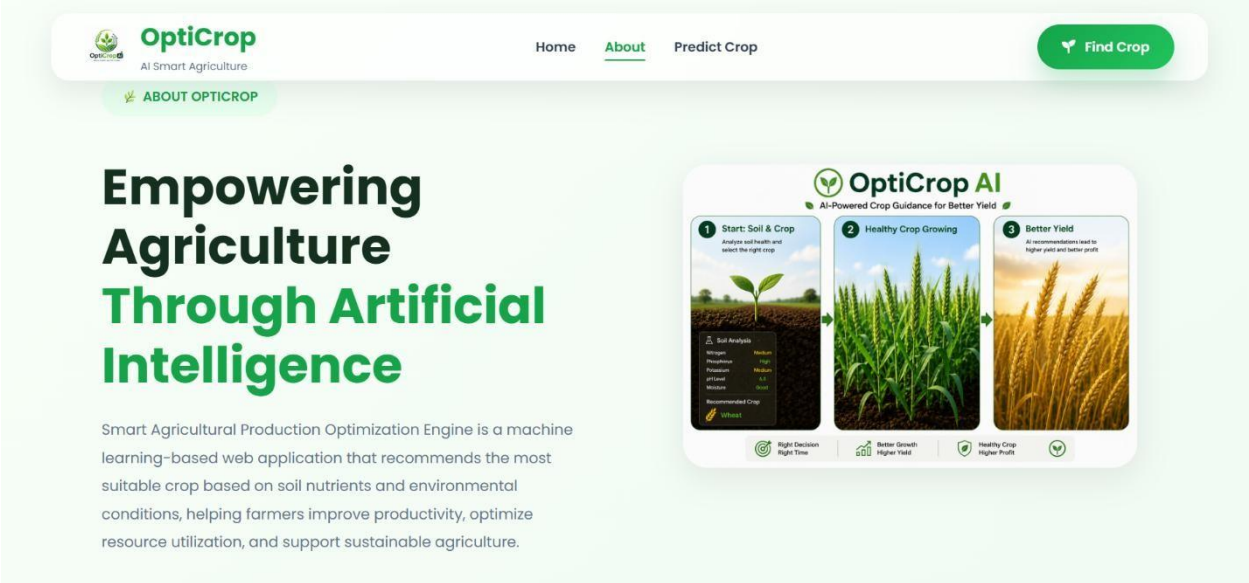

Figure 4.2: About page.

Predict Crop

The input form where users enter Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH Level, and Rainfall values, styled as a soil-sensor readout panel.

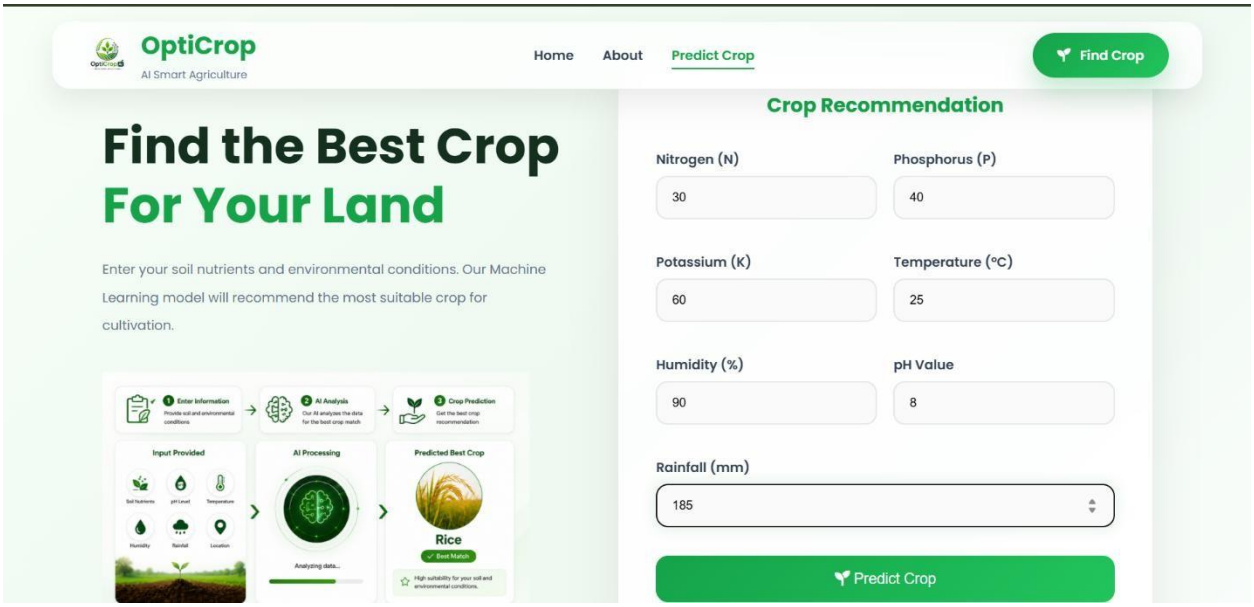

Figure 4.3: Find Your Crop page.

Prediction Result

Displays the recommended crop based on the entered soil and climate conditions. In this example, the model recommended Papaya for the given input values.

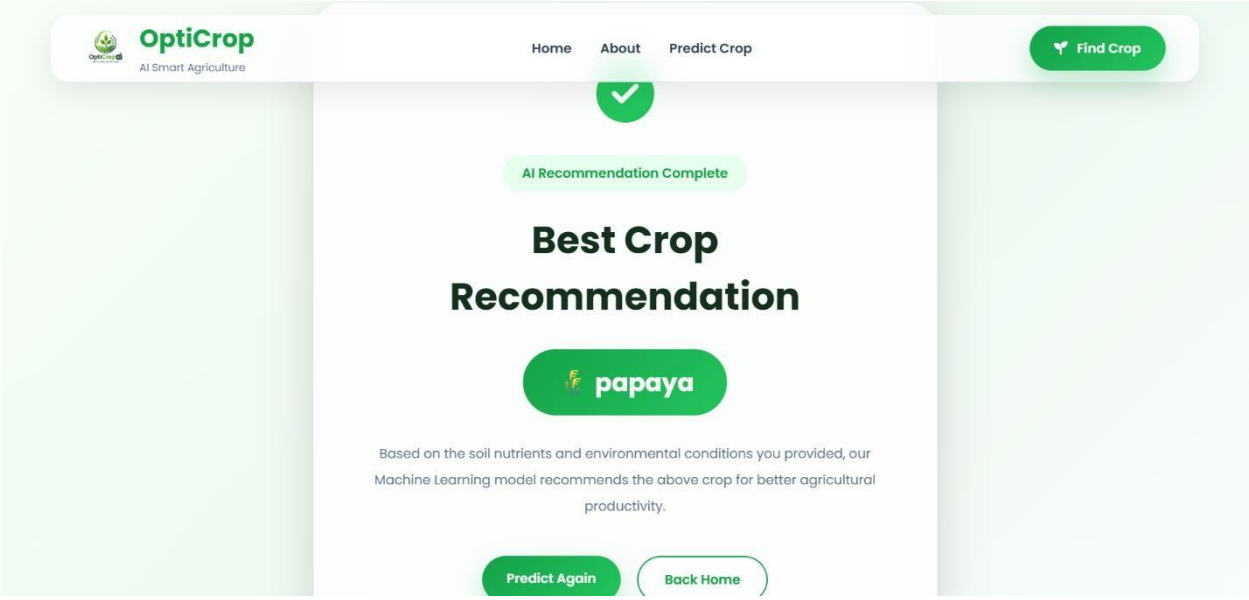

Figure 4.4: Prediction result page.

Testing

The application was manually tested with input profiles corresponding to known crops, verifying that predictions matched expected outcomes:

Expected Crop	N	P	K	Temp (°C)	Humidity (%)	pH	Rainfall (mm)	Predicted
Rice	90	42	43	20.8	82	6.5	202.9	Rice
Coffee	116	23	25	23.4	52.3	6.9	139.4	Coffee
Watermelon	80	26	55	24.5	89.0	6.1	49.1	Watermelon
Chickpea	32	73	81	20.5	15.4	6.0	92.7	Chickpea
Jute	100	35	36	25.3	72.0	6.3	190.6	Jute

Mango	20	19	35	34.2	50.6	6.1	98.0	Mango

Application-level testing also confirmed: form submission correctly sends all 7 parameters, the model successfully predicts the most suitable crop , the result page displays the predicted crop correctly, and navigation across Home / About / Predict Crop pages works as expected.

Local Deployment

- Clone the repository:
- Install dependencies: pip install -r requirements.txt
- Run the app: python app.py
- Access locally at: http://localhost:5000

Render Deployment

- Source control:
- Hosting: Render (free tier), deployed directly from the GitHub repository
- Build Command: pip install -r requirements.txt
- ● Start Command: python app
- Live URL:

Conclusion

The Smart Agricultural Production Optimization Engine was successfully developed using Python, Flask, and Machine Learning. The application accepts user inputs such as Nitrogen, Phosphorus, Potassium, temperature, humidity, pH, and rainfall, and predicts the most suitable crop with 97.3% accuracy. The system was tested with multiple input values corresponding to different crops, and the results were successfully displayed through the deployed Flask web application, accessible both locally and publicly via Render.

Future Scope

- Incorporate soil type as an additional input feature, requiring a richer dataset and model retraining.

- Add fertilizer and irrigation recommendations alongside crop suggestions.
- Support multiple regional languages for wider farmer accessibility.
- Integrate live weather API data to auto-fill temperature, humidity, and rainfall fields.
- Upgrade hosting to a paid tier to eliminate cold-start delays.

References

- Dataset: Kaggle — Smart Agricultural Production Optimizing Engine (chitrakumari25)
- Scikit-learn Documentation: https://scikit-learn.org/stable/
- Flask Documentation: https://flask.palletsprojects.com/ ● Project Repository:
- Live Application: